

Cahier des charges

Gestion de terrains de Padel

Paul MALIOUKOV - PSR10219

EPHEC Enseignement Supérieur de Promotion sociale

Année académique : 2025 - 2026

Unité d'enseignement : 562/1 IN3-Projet de développement SGBD

Responsable d'enseignement : M. Fievez Vincent

Unité associée au projet : 561/1 IN3-Projet développement Web

Responsable d'unité associée : M. Hardenne Romain

Projet cible : ProjetDV2026-PMALIOUKOV

Dépôt git : [ProjetDV2026_PMLKV](#)

Outils de développement :

1. IntelliJ IDEA 2026 IDE
2. Maven
3. Docker Desktop
4. Microsoft SQL Server 2022

Table des matières

1. Contexte	4
1.1. Portée	4
1.2. Projet	4
2. Analyse métier	5
2.1. Contexte client	5
2.1.1. Gestion des Sites	5
2.1.2. Gestion des réservations - règles globales.....	5
2.1.3. Gestion des réservations - règles événement privé	5
2.1.4. Gestion des annulations - règles événement privé	6
2.1.5. Gestion des réservations – pénalités événement privé	6
2.1.6. Gestion des réservations – règles événement public.....	6
2.1.7. Intervenants - utilisateurs	6
2.1.8. Interface - utilisateurs	7
2.1.9. Intervenants - administrateurs.....	7
2.1.10. Interface - administrateurs	7
2.2. Contexte de développement	7
2.2.1. Exigences fonctionnelles	9
2.2.2. Exigences non fonctionnelles	10
2.2.3. Contraintes business	11
3. Analyse fonctionnelle	13
3.1. Diagramme des cas d'utilisation	13
3.2. Description de 3 cas d'utilisation principaux	14
3.2.1. UC-1 Authentification.....	14
3.2.2. UC-2 Création d'un match privé.....	16
3.2.3. UC-3 Transformation d'un événement privé en public.....	20
3.3. Vues UC-1 et UC-2.....	23
3.3.1. Vue d'exemple pour - UC-1 Authentification.....	24
3.3.2. Vue d'exemple pour - UC-2 Création d'un match privé.....	26
3.4. Architecture applicative	31
3.4.1. Architecture Backend	31
3.4.2. Architecture Frontend	33
4. Contraintes fonctionnelles	35
4.1. Règles	35
4.1.1. Accès et autorisation.....	36

4.1.2.	Authentification et sécurité.....	37
4.1.3.	Gestion des événements (matches)	37
4.1.4.	Gestion des utilisateurs et invitations	38
4.1.5.	Paielements et abonnements	39
4.1.6.	Consultation et affichage	39
4.2.	Calculs et opérations complexes.....	40
4.2.1.	Calcul des sessions sur base d'horaires d'ouverture	40
4.2.2.	Génération du matricule.....	43
5.	Modélisation des données.....	45
5.1.	Description des entités	45
5.2.	Diagramme Entité-Association (EA)	46
5.3.	Diagramme relationnel.....	47
5.4.	Processus de conception	48
5.4.1.	Canvas d'analyse initiale	48
5.5.	Portée et évolution	49
5.5.1.	Tableau d'implémentation des contraintes	49
5.5.2.	Stratégie d'implémentation des contraintes.....	54
5.5.3.	Fonctionnalités reportées.....	54
6.	Conclusion	56
6.1.	Annexes.....	57

1. Contexte

1.1. Portée

Le projet « Gestion de terrains de Padel » s'inscrit dans le cadre des UE 562/1 (IN3-Projet de développement SGBD) et 561/1 (IN3-Projet développement Web). Il vise à démontrer notre capacité à :

- Comprendre les besoins fonctionnels et métiers du client.
- Formaliser ces besoins en règles de gestion exploitables.
- Modéliser une solution technique cohérente.
- Utiliser un système de collaboration et de suivi (GitHub).
- Documenter les processus, composants et flux de données.
- Produire un cahier des charges reflétant le travail réalisé.

Le cahier des charges s'inscrit dans le cadre de l'UE 562/1 (IN3-Projet de développement SGBD). Il a pour but de démontrer notre aptitude à mener une analyse projet.

1.2. Projet

Ce projet consiste à développer une solution SaaS de gestion entrepreneuriale pour des complexes sportifs multisites, offrant des terrains de Padel en intérieur et extérieur, avec une gestion des droits par rôles.

Chaque site dispose d'horaires spécifiques et d'un calendrier prédéfini de fermetures (incluant les jours fériés légaux, partagés entre tous les sites). Les réservations sont soumises aux droits et règles métiers définis au préalable.

La gestion des rôles, basée sur des identifiants uniques, doit s'appuyer sur une implémentation sécurisée garantissant la confidentialité des données côté client, des droits spécifiques par rôle et sur une authentification par jeton unique à durée limitée.

La plateforme devra proposer l'organisation de matchs publics, de matchs privés et un système de gestion de paiement précis. Des règles visant à gérer les événements annulés ou incomplets devront également être implémentés, incluant un système de pénalités associé.

Une base de données relationnelle est indispensable à la solution pour extraire et mettre à jour uniquement les données nécessaires, tout en évitant les suppressions arbitraires et en garantissant des opérations propres.

2. Analyse métier

2.1. Contexte client

2.1.1. Gestion des Sites

Chaque site du complexe dispose des paramètres spécifiques suivants :

- Nombre de terrains : Variable selon le site.
- Horaires d'ouverture : Propres à chaque site.
- Plage de réservation : Heure de début et de fin des réservations, définie par site.
- Jours de fermeture : Partagés et propres à chaque site.

2.1.2. Gestion des réservations - règles globales

Durée d'une réservation : 1h30 (90 minutes), incluant un intervalle de 15 minutes entre chaque événement (match).

Calendrier applicable : Les horaires sont valables pour une année civile, avec possibilité de révision pour l'année suivante. Intégration des jours fériés et fermetures annuelles prédéfinis.

Coût : 60,00 € par événement (match), à régler intégralement à l'avance.

Joueurs : 4 joueurs (max/min)

2.1.3. Gestion des réservations - règles événement privé

Règles d'une réservation : Réservation obligatoire 48h (min) à l'avance (sous réserve de disponibilités). L'organisateur doit enregistrer les participants (4 joueurs max/min).

Règles paiement : Les modalités de paiement stipulent que chaque joueur est tenu de régler sa part individuelle, soit un quart (1/4) du coût total du match. Aucun solde impayé n'est autorisé pour les participants confirmés, sous peine de pénalité pour l'organisateur.

Règles d'affichage : N/A.

2.1.4. Gestion des annulations - règles événement privé

Si un ou plusieurs joueurs ne paient pas leur part ou ne confirment pas leur participation :

- Le match devient public.
- La place du / des joueurs est libérée et réservable par un autre utilisateur.
- Le solde restant dû reste exigible (à la charge de l'organisateur).

2.1.5. Gestion des réservations – pénalités événement privé

Si un solde restant dû est exigible, l'intervenant en question se voit infliger un blocage des réservations jusqu'au règlement du solde.

Si le nombre de joueurs n'est pas atteint lors d'un événement (match) privé dû à un défaut de paiement, refus ou non-confirmation par un joueur, l'organisateur se voit infliger une pénalité de sept jours ouvrables.

2.1.6. Gestion des réservations – règles événement public

Règles d'une réservation : Réservation obligatoire 24h (min) à l'avance (sous réserve de disponibilités). Les participants ne doivent avoir aucun solde impayé et il n'est pas permis d'inviter des joueurs à un match public (inscriptions individuelles).

Règles paiement : Les modalités de paiement stipulent que chaque joueur est tenu de régler sa part individuelle, soit un quart ($\frac{1}{4}$) du coût total du match, à l'inscription.

Règles d'affichage : Visibilité publique des places disponibles.

2.1.7. Intervenants - utilisateurs

Les utilisateurs sont classés en trois catégories, identifiées par un matricule unique :

Type	Matricule	Avantages	Délai de réservation (max jours)
Invité / Libre	L0000	/	5
Membre	S0000	Accès libre au site souscrit.	14
VIP / Global	G0000	Accès libre à tous les sites.	21

2.1.8.Interface - utilisateurs

L'interface utilisateur permet la consultation et la gestion des réservations (publiques/privées). Dépendamment du rôle, l'intervenant pourra explorer une vue par site ou une vue globale (tous les sites avec filtrage).

2.1.9.Intervenants - administrateurs

Les administrateurs sont classés en deux catégories, identifiés par un matricule unique :

Type	Matricule	Avantages
Super Administrateur	A000	Administration de tous les sites.
Administrateur Site	M000	Administration d'un site.

2.1.10. Interface - administrateurs

L'interface administrateur permet la consultation et la gestion des horaires / fermetures, inscriptions, matchs et terrains. Dépendamment du rôle, l'intervenant pourra explorer une vue par site ou une vue globale (tous les sites avec filtrage). Celle-ci inclut également un suivi du chiffre d'affaires et des statistiques.

2.2. Contexte de développement

Ce projet adopte une approche « Database-First », une méthodologie où la conception de la base de données précède le développement de l'application. Cette approche présente plusieurs avantages clés pour un système de gestion comme celui des terrains de Padel :

a) Modélisation rigoureuse des données.

Les relations complexes (sites, terrains, réservations, utilisateurs, rôles, paiements) sont d'abord structurées dans le schéma de la base de données, garantissant une cohérence logique avant même d'écrire une ligne de code applicatif.

Exemple : la gestion des créneaux horaires par site, des réservations privées/publiques ou des pénalités reposent sur un modèle de données normalisé et cohérent.

b) Intégrité et fiabilité.

Les clés étrangères (REFERENCES) assurent la cohérence entre les tables (ex. : *une réservation ne peut exister sans terrain associé*).

Les contraintes CHECK enforcent les règles métiers directement : :

- *check_match_time : end_time > start_time (durée valide pour un match).*
- *check_match_status : Un match ne peut être à la fois privé et public.*
- *check_user_status : status IN ('clear', 'debt') (statut financier d'un utilisateur).*

Les contraintes UNIQUE évitent les doublons (ex. : *uq_user_site pour un utilisateur sur un site*).

c) Flexibilité pour l'application.

La base de données définit les règles de base (ex. : *min_players = 4*), tandis que la logique complexe (ex. : *conversion d'un match privé en public en cas de solde impayé*) est gérée par l'application.

Exemple : la table de paiements liés aux matchs utilise des contraintes pour valider le statut (clear, pending, etc.), mais c'est l'application qui déclenche les actions, comme appliquer une pénalité.

Cette approche est idéalement adaptée à un système comme celui-ci, où les données sont critiques (réservations, paiements, rôles) et les règles métier évoluent (ex. : *ajout de nouveaux types de pénalités*). Elle permet une évolutivité tout en maintenant une structure de données solide et maintenable.

2.2.1.Exigences fonctionnelles

Les fonctionnalités applicatives attendues sont classées dans le tableau ci-dessous par niveau de priorité (1 = élevée, 5 = basse) et par intervenant. Une colonne « Implémenté » permet de suivre leur déploiement en production (X = implémenté, P = partiellement implémenté, N = à implémenter).

Fonctionnalité	Intervenant	Description	P	Imp ?
S'authentifier sur la plateforme.	Utilisateur / Administrateur	Un intervenant doit pouvoir s'authentifier de manière sécurisée sur la plateforme.	1	X
Consulter les sites et terrains disponibles, ainsi que leurs créneaux horaires.	Utilisateur / Administrateur	Un intervenant doit pouvoir consulter, avant toute réservation de match, les sites et terrains disponibles, ainsi que leurs créneaux horaires (libres, occupés ou fermetures).	1	P
S'enregistrer en tant qu'utilisateur.	Utilisateur	Un utilisateur doit pouvoir s'enregistrer sur la plateforme.	1	X
Rejoindre un match public.	Utilisateur	Un utilisateur doit pouvoir rejoindre un match public.	1	X
Créer un match privé.	Utilisateur	Un utilisateur doit pouvoir créer un match privé et y inviter des joueurs.	1	X
Payer une participation.	Utilisateur	Un utilisateur doit pouvoir payer sa participation à un match.	1	X
Création d'un match public par administrateur.	Administrateur	Un administrateur doit pouvoir créer des matchs publics pour le / les complexes sportifs proposés.	1	X
Gestion du profil.	Utilisateur / Administrateur	Un intervenant doit pouvoir consulter son profil et y faire les changements nécessaires (changement de mot de passe, changement de données, etc.)	2	P
Déconnexion.	Utilisateur / Administrateur	Un intervenant doit pouvoir se déconnecter, de manière sécurisée, de la plateforme.	2	X
Gestion sur base du rôle.	Utilisateur / Administrateur	Un intervenant doit uniquement avoir accès aux éléments disponibles pour son rôle.	2	X
Inviter un joueur / nouveau membre.	Utilisateur	Un utilisateur doit pouvoir inviter un joueur existant ou nouveau membre lors de la création d'un match privé.	2	X
Vue des matchs à venir.	Utilisateur	Un utilisateur doit pouvoir consulter ses matchs à venir.	2	X
Vue des invitations.	Utilisateur	Un utilisateur doit pouvoir visualiser et accepter / décliner ses invitations à des matchs privés.	2	X
Gestion des dettes / pénalités.	Utilisateur	Un utilisateur doit pouvoir consulter ses pénalités et apurer ses dettes.	2	X
Mise à niveau de l'abonnement.	Utilisateur	Un utilisateur doit pouvoir mettre à jour son abonnement (utilisateur invité → utilisateur site ; utilisateur site → VIP).	3	P

Gestion des sites, terrains et matchs.	Administrateur	Un administrateur doit pouvoir mettre à jour, ajouter ou modifier les données d'entités ou des matchs existants.	3	P
Gestion des utilisateurs.	Administrateur	Un administrateur doit pouvoir désactiver, ou modifier les informations, d'utilisateurs existants.	4	P
Création d'un match public par utilisateur.	Utilisateur	Un utilisateur doit pouvoir créer un match public sur le /les complexes sportifs disponibles.	5	N
Gestion des statistiques et du chiffre d'affaires.	Administrateur	Un administrateur doit pouvoir consulter les statistiques et les données financières d'une entité.	5	N

2.2.2.Exigences non fonctionnelles

Le projet doit respecter des exigences non fonctionnelles, qui définissent la qualité et les attentes de fonctionnement (et non les fonctionnalités). Ces exigences sont classées dans le tableau ci-dessous par niveau de priorité (1 = élevée, 5 = basse. Une colonne « Implémenté » permet de suivre leur déploiement en production (X = implémenté, P = partiellement implémenté, N = à implémenter).

Exigence	Description	P	Imp ?
Intégrité des données.	Les données enregistrées doivent rester cohérentes (contraintes d'intégrité, clés primaires, clés étrangères, checks, etc.)	1	X
Cohérence des données reçues.	Les données reçues doivent rester cohérentes.	1	X
Contrôle des permissions.	Une gestion des accès et permissions doit être implémentée au niveau applicatif, tout comme au niveau de la base de données. Les opérations autorisées doivent être clairement différenciées selon les rôles.	1	X
Sécurité.	L'authentification doit respecter les standards de sécurité et les données sensibles doivent être hachées / encryptées.	1	X
Historique et traçabilité.	Les opérations critiques sont datées et liées aux entités concernées.	1	X
Gestion des erreurs.	Gestion des erreurs.	2	X
Gestion de la concurrence.	Prévenir les conflits d'accès (ex. : double réservation du même créneau).	2	X
Extensibilité.	Le modèle relationnel de la base de données doit rester structuré et extensible.	2	X
Maintenabilité des données.	Les données utilisées par la logique métier doivent pouvoir être modifiées dynamiquement sans interrompre l'exécution de la solution.	2	P

Maintenabilité applicative.	La solution doit adopter une architecture en trois couches (présentation, logique métier, accès aux données) pour assurer une séparation claire des responsabilités, une maintenabilité accrue et une évolutivité optimale, tout en mettant en œuvre des méthodologies rigoureuses de commentaires, de logging et de documentation.	2	X
Testabilité.	La solution doit inclure des tests exécutables sans altérer ni interrompre le fonctionnement du système.	3	P
Compatibilité.	Design « responsive » pour une utilisation optimale sur chaque type d'appareil.	3	X
Performance.	La solution doit éviter le code répétitif et les appels API redondants pour optimiser les performances et l'interactivité.	4	X
Accessibilité et lisibilité.	L'interface générale doit garantir une lisibilité et une accessibilité optimales pour tous les utilisateurs.	4	X
Portabilité.	Le code doit être portable entre environnements (ex. : docker).	5	X
Locale.	L'interface doit supporter le multilingue.	5	N
Disponibilité.	La solution doit être accessible via un domaine dédié.	5	N

2.2.3.Contraintes business

Les principales contraintes métier du projet sont reprises dans le tableau ci-dessous. Celles-ci ont un impact direct sur la gestion des données et les processus applicatifs.

Contrainte	Description	Action
Gestion des créneaux horaires par site.	Chaque site dispose d'horaires d'ouverture et de fermeture fixes. Il est essentiel d'intégrer les matchs sans imposer aux joueurs des temps d'attente excessifs, chaque match durant 1h30 (90 min) (durée non modifiable).	Restreindre les types d'horaires autorisés et calculer automatiquement les sessions pour inclure 15 à 30 minutes de marge avant la première et après la dernière session.
Fermetures.	Les jours de fermeture peuvent être applicable à un ou plusieurs sites.	Créer une séparation entre les deux types (applicable à un site / multisite) tout en minimisant le nombre d'entrées dans la base de données.
Gestion de plusieurs entités.	L'environnement est constitué de plusieurs sites, chacun avec ses propres terrains, horaires et fermetures spécifiques.	Assurer une distinction entre les différentes entités.
Gestion des utilisateurs.	Le système distingue cinq (5) rôles utilisateur (deux (2) administrateurs et trois (3) utilisateurs).	Implémenter un contrôle d'accès pour gérer les rôles et restreindre les actions selon le profil.

Gestion des matricules utilisateur.	Les rôles utilisateur sont identifiés par un matricule unique (en plus du role_id), permettant une distinction claire entre les types de comptes.	Automatiser la génération des matricules en fonction du rôle de l'utilisateur (ex. : préfixe L pour les invités, S pour les membres site, etc.).
Transfert de rôle utilisateur.	Le rôle d'un utilisateur peut être modifié (ex. : membre site → membre VIP).	Valider que le transfert est autorisé : pas de rétrogradation dans la hiérarchie des rôles, et pas de transfert entre rôles utilisateur (0-2) et rôles administrateur (7-9). S'assurer également que le matricule utilisateur est mis à jour.
Validation des contraintes événementielles.	Les délais de réservation varient selon le rôle utilisateur. Chaque match doit comporter exactement 4 joueurs, et les matchs privés doivent être restreints aux joueurs invités.	Implémenter des validations pour : appliquer les délais par rôle, vérifier le nombre de joueurs, et limiter l'accès aux matchs privés aux invités.
Durée d'un événement.	Un événement (match) doit durer 1h30 (90 min). Chaque événement doit être séparé par des intervalles de 15 minutes.	Eviter les chevauchements et prévoir un contrôle des créneaux horaires par événement.
Conversion automatique des événements privés en publics.	Si un événement privé est incomplet (défaut de paiement ou de confirmation) la veille de son déroulement, il doit être automatiquement converti en événement public.	Mettre en place un scheduler (cron) pour détecter les événements privés incomplets et les convertir en publics, tout en libérant les places non confirmées.
Validation des paiements.	Chaque joueur doit effectuer son paiement pour participer à l'événement, sous peine d'annulation. Le montant s'élève à un quart (1/4) du prix du match (60,00 €).	Enregistrer systématiquement la participation et le paiement de chaque joueur dans la base de données. Valider le montant du /des paiements.
Gestion des pénalités.	Une pénalité est appliquée en cas de : défaut de paiement, absence non justifiée ou non-confirmation d'un événement. Cette pénalité est exclusivement appliquée à l'organisateur de l'événement.	Implémenter une gestion des pénalités et enregistrer dans la base de données les informations associées (raison, durée, montant, etc.). Mettre en place un scheduler (cron) pour mettre à jour les pénalités expirées.
Statistique.	Suivi et analyse des données.	La base de données doit permettre l'exploitation des paiements, événements et participations à des fins d'analyse.

3. Analyse fonctionnelle

3.1. Diagramme des cas d'utilisation

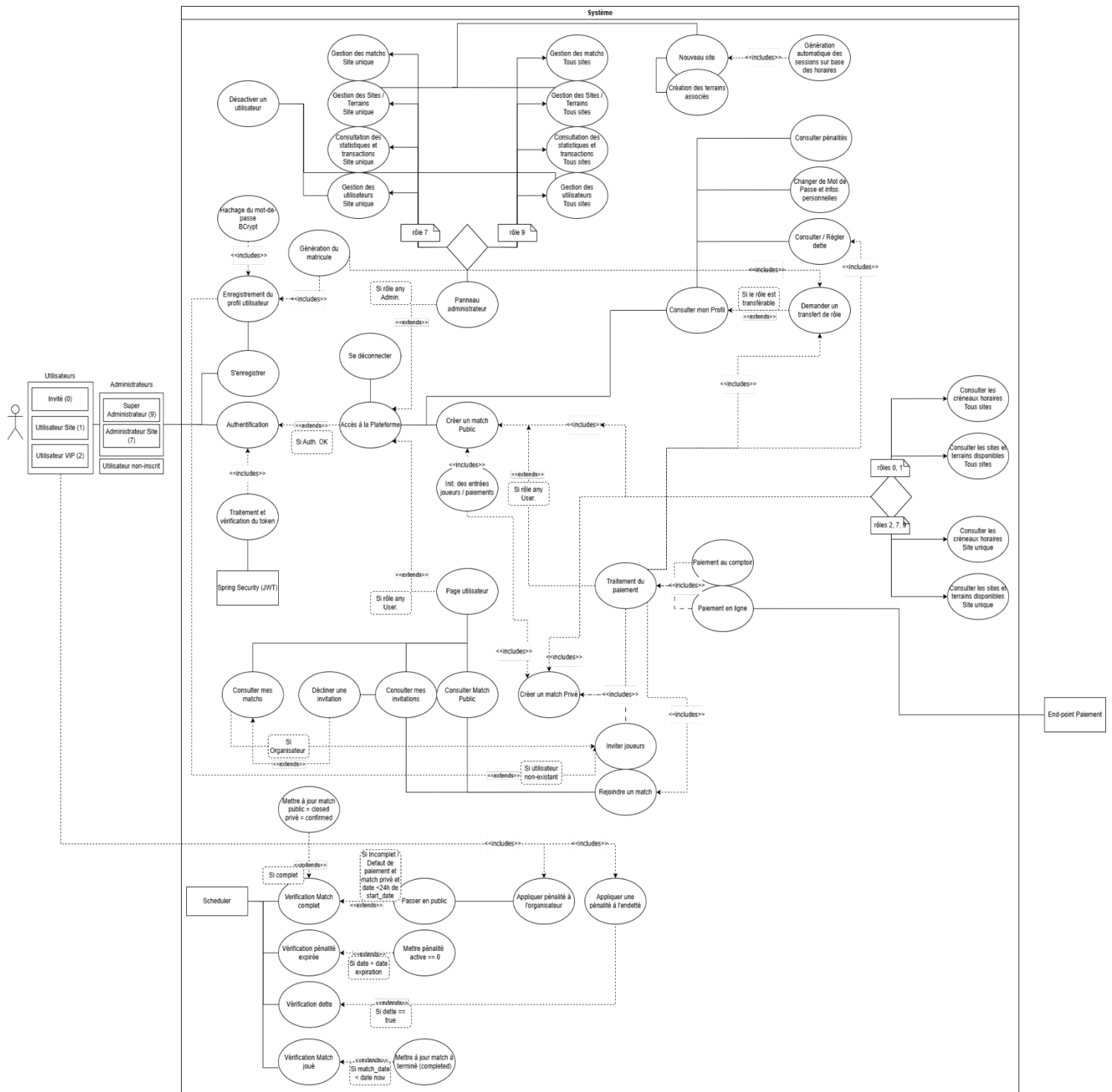


Figure 1 - Diagramme UML des cas d'utilisation

3.2. Description de 3 cas d'utilisation principaux

Les cas d'utilisation suivants décrivent de manière détaillée trois (3) fonctionnalités principales du système pour mieux comprendre le diagramme des cas d'utilisation ([page 13](#)).

3.2.1.UC-1 Authentification

Objectif :

Permettre à un intervenant de se connecter à la plateforme.

Acteur(s) :

- **Utilisateurs** :
 - Invité (rôle 0)
 - Utilisateur Site (rôle 1)
 - Utilisateur Tous Sites, VIP (rôle 2)
- **Administrateurs** :
 - Administrateur Site (rôle 7)
 - Administrateur Tous Sites, SA (rôle 9)
- **Autres** :
 - Spring Security (JWT)

Prérequis :

L'intervenant a :

- Un appareil supporté (PC, mobile ou tablette).
- Accès à la page d'accueil de l'application web.
- Un identifiant valide présent dans le système.

Action :

L'intervenant souhaite s'authentifier sur la plateforme.

Vue d'exemple : [pages 24 - 25](#)

Scénario nominal :

1. L'intervenant accède à la page d'accueil de la plateforme.
2. Il clique sur le bouton « Se connecter » situé dans l'angle supérieur droit.
3. Un email valide présent dans le système est introduit.
4. Un mot-de-passe correspondant est introduit.
5. Le système invoque l'*AuthenticationManager* de Spring Security.
6. L'*AuthenticationManager* utilise un service qui vérifie si l'utilisateur existe bien dans la base de données et que les paramètres de connexion correspondent aux paramètres hachés existants.
7. Le service JWT génère un jeton signé avec expiration (900s) et le renvoie.
8. Le jeton JWT reçu est sauvegardé dans le cache du navigateur jusqu'à expiration.
9. L'utilisateur est redirigé vers la page utilisateur / panneau administration.

Scénario alternatif 1 - identifiants incorrects :

1. L'intervenant accède à la page de connexion.
2. Il introduit un email valide mais un mot de passe incorrect.
3. Le système tente de l'authentifier via l'*AuthenticationManager*.
4. La vérification du mot de passe échoue.
5. Le système rejette la tentative d'authentification et renvoie une erreur *401 Unauthorized*.
6. L'interface utilisateur affiche un message d'erreur à l'utilisateur, tel que "Email ou mot de passe incorrect".

Scénario alternatif 2 - utilisateur non trouvé :

1. L'intervenant accède à la page de connexion.
2. Il introduit un email qui n'existe pas dans la base de données.
3. Le système ne trouve aucun utilisateur correspondant.
4. Le service lève une exception *UsernameNotFoundException*.
5. Le système renvoie une erreur *401 Unauthorized* (pour ne pas donner d'indices sur l'existence ou non d'un utilisateur).
6. L'interface utilisateur affiche un message d'erreur à l'utilisateur, tel que "Email ou mot de passe incorrect".

Scénario alternatif 3 - compte utilisateur inactif :

1. L'intervenant accède à la page de connexion.
2. Il introduit des identifiants valides pour un compte qui a été désactivé (is_active = 0 en base de données).
3. Le système vérifie que le compte existe mais est marqué comme inactif.
4. Spring Security rejette l'authentification (compte désactivé).
5. Le système renvoie une erreur *403 Forbidden*.
6. L'interface utilisateur affiche un message d'erreur à l'utilisateur, tel que "Email ou mot de passe incorrect".

3.2.2.UC-2 Création d'un match privé

Objectif :

Permettre à un intervenant de réserver un terrain sur un site, à une date et un créneau horaire précis, en tant qu'événement (match) privé, et d'y inviter 3 autres utilisateurs (existants ou non).

Acteur(s) :

- **Utilisateurs :**
 - Invité (rôle 0)
 - Utilisateur Site (rôle 1)
 - Utilisateur Tous Sites, VIP (rôle 2)
- **Autres :**
 - End-point Paiement

Prérequis :

L'intervenant doit :

- Être connecté (jeton JWT valide).
- Ne pas avoir de pénalités et/ou de dettes.
- Avoir un rôle autorisé (0, 1 ou 2).
- Avoir un site assigné.

Action :

L'intervenant souhaite créer un événement (match) privé.

Vue d'exemple : [pages 26-30](#)

Scénario nominal :

1. À partir de la page utilisateur, l'intervenant sélectionne « Créer Match Privé » dans le menu de navigation.
2. Le système vérifie si l'intervenant n'a pas de pénalités actives / dettes impayées.
3. Un formulaire s'ouvre pour réserver un événement (match) privé.
4. Il sélectionne un site (complexe sportif) parmi ceux auxquels il a accès.
5. Il sélectionne un terrain parmi ceux actifs et disponibles du site.
6. Il sélectionne une date disponible via le module calendrier, en vérifiant :
 - a. Le site n'est pas fermé ce jour-là (requête sur les fermetures).
 - b. Aucun créneau n'est totalement réservé (requête sur les matchs).
 - c. La date est dans sa zone de réservation (ex. : règle métier pas de réservation > X jours à l'avance).
7. L'intervenant sélectionne l'heure du début de l'événement à partir de la liste des créneaux disponibles endéans les horaires d'ouverture.
8. Une heure de fin est automatiquement assignée marquant la fin de l'événement (+ 90 minutes) informant l'intervenant.
9. L'intervenant invite 3 joueurs :
 - a. Pour chaque invité, il saisit un e-mail (existant ou non) et clique sur l'icône de recherche.
 - b. Le système valide le format de l'e-mail.
 - c. Le système vérifie :
 - i. L'existence de l'utilisateur.
 - ii. Que l'utilisateur invité n'a pas de pénalités actives, est actif ou n'est pas administrateur.
 - iii. Que l'utilisateur invité n'a pas d'événements existants à cette date / créneau horaire.
 - iv. Que l'utilisateur invité est différent de l'organisateur (intervenant).
 - v. Que les utilisateurs invités sont différents les uns des autres.
 - d. Sur base des validations :
 - i. Si l'utilisateur n'existe pas → l'interface utilisateur affiche un bouton « Inviter ? » :

Si l'utilisateur clique sur « Inviter ? » un formulaire s'affiche pour inscrire un utilisateur :

 - L'intervenant doit fournir le : prénom, nom, date de naissance de l'invité et le niveau.
 - L'intervenant doit soumettre le formulaire.
 - L'invité est créé dans le système avec un identifiant unique (matricule).

- ii. Si l'utilisateur existe ou est enregistré via le formulaire → l'interface utilisateur affiche un message « Utilisateur trouvé ».
- 10.** Le système vérifie que tous les champs du formulaire « Créer Match Privé » sont correctement remplis.
- 11.** Le bouton « Créer le Match » est désormais disponible.
- 12.** L'intervenant clique sur « Créer le Match ».
- 13.** Une invitation de paiement est présentée (¼ du coût total du match).
- 14.** L'intervenant introduit son numéro de carte et valide le formulaire :
 - a. Le paiement est validé par le end-point paiement.
 - b. Le système crée l'événement (match) privé (priv_status = 'awaiting'), crée les entrées pour les paiements (status = 'pending') des invités et enregistre les invités comme joueurs (status = 'pending') pour ce match.
 - c. Le paiement de l'organisateur (intervenant) est confirmé (status = 'clear', payment_method = 'CARD', payment_date = date_time_now).
 - d. La participation de l'organisateur est confirmée (role = p1, status = approved).
- 15.** L'interface présente un pop-up à l'intervenant qui confirme la réservation et que les utilisateurs invités ont une demande de paiement en attente.
- 16.** L'utilisateur clique sur « OK » et est redirigé vers la page utilisateur.

Scénario alternatif 1 – l'intervenant a une pénalité / dette en cours :

- 1.** À partir de la page utilisateur, l'intervenant sélectionne « Créer Match Privé » dans le menu de navigation.
- 2.** Le système a détecté une pénalité active et / ou une dette en cours.
- 3.** L'interface présente un pop-up à l'intervenant qui indique que celui-ci a une pénalité active ou une dette impayée, et affiche les informations relatives à la dette et/ou à la pénalité.

Scénario alternatif 2 – l'intervenant invite un utilisateur avec une pénalité / dette en cours :

Ce scénario alternatif concerne la neuvième (9) étape.

1. L'intervenant introduit un utilisateur avec une pénalité et / ou une dette.
2. Le système a détecté que l'utilisateur invité a une pénalité ou une dette active.
3. L'interface affiche un message d'erreur « Vous ne pouvez pas inviter cet utilisateur ».

Scénario alternatif 3 – l'intervenant invite un administrateur :

Ce scénario alternatif concerne la neuvième (9) étape.

1. L'intervenant introduit un utilisateur avec le rôle administrateur (7,9).
2. Le système a détecté que l'utilisateur invité est un administrateur.
3. L'interface affiche un message d'erreur « Impossible d'inviter un administrateur ».

Scénario alternatif 4 – l'intervenant s'invite lui-même :

Ce scénario alternatif concerne la neuvième (9) étape.

1. L'intervenant s'introduit lui-même.
2. Le système a détecté que l'utilisateur invité a le même identifiant que l'organisateur (intervenant).
3. L'interface affiche un message d'erreur « Vous ne pouvez pas vous inviter vous-même ».

Scénario alternatif 5 – l'intervenant invite deux fois le même utilisateur :

Ce scénario alternatif concerne la neuvième (9) étape.

1. L'intervenant introduit un utilisateur déjà présent dans le formulaire.
2. Le système a détecté que cet utilisateur a déjà été introduit.
3. L'interface affiche un message d'erreur « Les adresses doivent être différentes les unes des autres ».

Scénario alternatif 6 – l’intervenant invite un utilisateur avec un match existant à la même date / même créneau :

Ce scénario alternatif concerne la neuvième (9) étape.

1. L’intervenant introduit un utilisateur avec un événement (match) existant à la même date / même créneau.
2. Le système a détecté que l’utilisateur invité a un événement existant à cette date / créneau horaire.
3. L’interface affiche un message d’erreur « Ce joueur a déjà un match à cette heure ».

3.2.3.UC-3 Transformation d’un événement privé en public

Objectif :

Vérifier automatiquement si un match privé est incomplet ou en défaut de paiement à moins de 24h de sa date. Le convertir en match public, libérer les places non confirmées, et appliquer une pénalité ainsi qu’une dette à l’organisateur.

Acteur(s) :

- **Autres :**
 - Scheduler

Prérequis :

Le match doit :

- Exister.
- Avoir une date inférieure à 24h avant le match_date (ex. : si match le 10/05, vérification le 09/05 à partir de minuit).
- Ne pas être confirmé : au moins un joueur a un status = 'pending' / 'declined' ou un paiement avec un status != 'clear'.

Action :

Pour chaque match identifié, le système convertit celui-ci en public en mettant à jour les champs type à 'public', pub_status à 'open' et priv_status à NULL dans la table des matchs. Il libère ensuite les places des joueurs non confirmés en supprimant les entrées correspondantes dans la table des joueurs où status = 'pending' ou 'declined'.

Une pénalité est appliquée à l'organisateur par insertion :

- reason = 'insufficient_players', start_date = NOW(), end_date = NOW() + 7 jours et is_active = 1.

Une dette est également appliquée au compte de l'organisateur en mettant à jour :

- balance = (balance - (1/4 du prix total du match * invités non-confirmés ou défaut de paiement)) et status = 'debt'.

L'organisateur et les joueurs ayant déjà confirmé / payé leur participation sont conservés pour le match public.

Scénario nominal :

- 1.** Le Scheduler (planificateur de tâches) s'exécute automatiquement toutes les 5 minutes.
- 2.** Le système recherche tous les matchs privés dont le statut est 'awaiting' et dont la date est fixée au lendemain (NOW() + 1 jour).
- 3.** Pour chaque match correspondant aux critères, le système exécute les actions suivantes :
 - a. Le statut du match privé original est mis à jour à 'cancelled'.
 - b. Un nouveau match est créé avec le type public et le statut open, en conservant les mêmes informations (terrain, date, heure, prix, etc.).
 - c. Les joueurs du match privé original qui avaient un statut 'approved' sont automatiquement inscrits au nouveau match public avec le même statut.
 - d. Les paiements déjà effectués ('clear') par les joueurs confirmés sont transférés et associés au nouveau match public.
 - e. Les places des joueurs qui n'avaient pas confirmé ('pending' ou 'declined') sont libérées dans le nouveau match public (elles deviennent des places avec le statut 'pending' et sans utilisateur assigné).
- 4.** Le système calcule une dette pour l'organisateur. Cette dette correspond au montant total des places non payées ou non confirmées par ses invités.
- 5.** Le solde (balance) du compte de l'organisateur est mis à jour pour refléter cette nouvelle dette, et son statut de compte passe à 'debt'.

6. Une pénalité est créée et appliquée à l'organisateur :

Motif : insuffisant_players
Date de début : NOW() (maintenant)
Date de fin : NOW() + 1 semaine
Statut : is_active = true

7. Le système enregistre toutes les modifications dans la base de données et journalise la conversion réussie du match.

Scénario alternatif 1 – aucun match privé ne remplit les conditions de conversion :

- 1.** Le Scheduler s'active toutes les 5 minutes pour exécuter la tâche de conversion.
- 2.** Le système recherche dans la base de données les matchs privés avec le statut priv_status = 'awaiting' et dont la date (match_date) est fixée au lendemain.
- 3.** Le système ne trouve aucun match correspondant à ces critères.
- 4.** Le Scheduler termine son exécution sans effectuer aucune modification. Un message est consigné dans les logs indiquant qu'aucun match n'a été converti.

Scénario alternatif 2 – erreur technique durant le traitement du match :

- 1.** Le Scheduler identifie un match privé à convertir et commence le processus.
- 2.** Une erreur technique se produit (par exemple, lors de la sauvegarde du nouveau match public).
- 3.** La transaction pour ce match uniquement est annulée (ROLLBACK). Toutes les modifications (annulation du match privé, création du match public, etc.) sont invalidées. Le match privé reste donc en l'état 'awaiting' jusqu'à la prochaine exécution.
- 4.** Une erreur (ERROR) est consignée dans les logs, détaillant le problème et l'ID du match concerné.
- 5.** Le Scheduler n'est pas interrompu et poursuit son exécution pour traiter les autres matchs de la liste.

Scénario alternatif 3 – l'organisateur du match n'est pas le joueur principal :

- 1.** Le Scheduler identifie un match privé à convertir.
- 2.** Le système procède à la conversion : il annule le match privé et crée un nouveau match public, en conservant les joueurs confirmés.
- 3.** Au moment de calculer la dette, le système vérifie si le matricule de l'organisateur (match.organiser) correspond au matricule de l'utilisateur dans le slot playerRole = 'p1'.
- 4.** La vérification échoue.
- 5.** Le système n'applique ni la dette ni la pénalité à l'organisateur.
- 6.** Un avertissement (WARN) est enregistré dans les logs pour signaler l'incohérence et le fait que la dette et la pénalité n'ont pas été appliquées.
- 7.** Le processus se termine pour ce match, et le Scheduler passe au suivant.

3.3. Vues UC-1 et UC-2

Pour illustrer de manière concrète les cas d'utilisation UC-1 (Authentification) et UC-2 (Création d'un match privé), il est utile d'y associer des vues d'exemple montrant les informations affichées à l'utilisateur. Cette approche permet d'obtenir une analyse plus complète et visuelle.

Note : Le UC-3 (Transformation d'un événement privé en public) n'est pas inclus, car il s'agit d'un cas d'utilisation réservé au système (exécuté par le Scheduler), sans interaction directe avec l'utilisateur.

3.3.1. Vue d'exemple pour - UC-1 Authentification

1. Page d'accueil de la plateforme :



Pourquoi venir chez nous ?

Rejoignez de nombreux sportifs déjà inscrits chez nous et
plongez dans l'univers passionnant du padel!
Que vous soyez débutant ou joueur confirmé, nos **16 terrains de qualité
professionnelle** vous attendent pour des parties intenses et conviviales.

Figure 2.1 - Page d'accueil de l'application web

2. Bouton « Se connecter » :

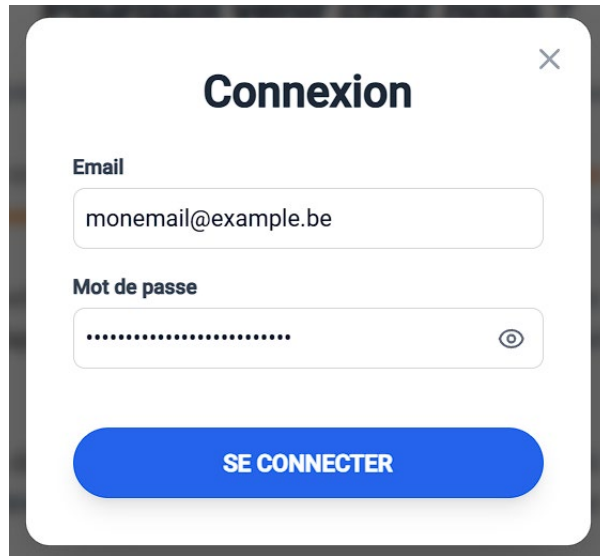
- PC / Tablette :



- Mobile :



3. Formulaire de connexion :



The image shows a login form titled "Connexion" in a dark blue font. The form is enclosed in a white box with rounded corners and a dark blue border. In the top right corner of the box is a small "X" icon. Below the title, there are two input fields. The first is labeled "Email" and contains the text "monemail@example.be". The second is labeled "Mot de passe" and contains a series of dots, with a small eye icon to its right. Below these fields is a large, rounded blue button with the text "SE CONNECTER" in white capital letters.

Figure 2.2 - Formulaire de connexion

3.3.2. Vue d'exemple pour - UC-2 Création d'un match privé

1. Page utilisateur post-authentification (rôle utilisateur) :
 - Sur PC / Tablette :

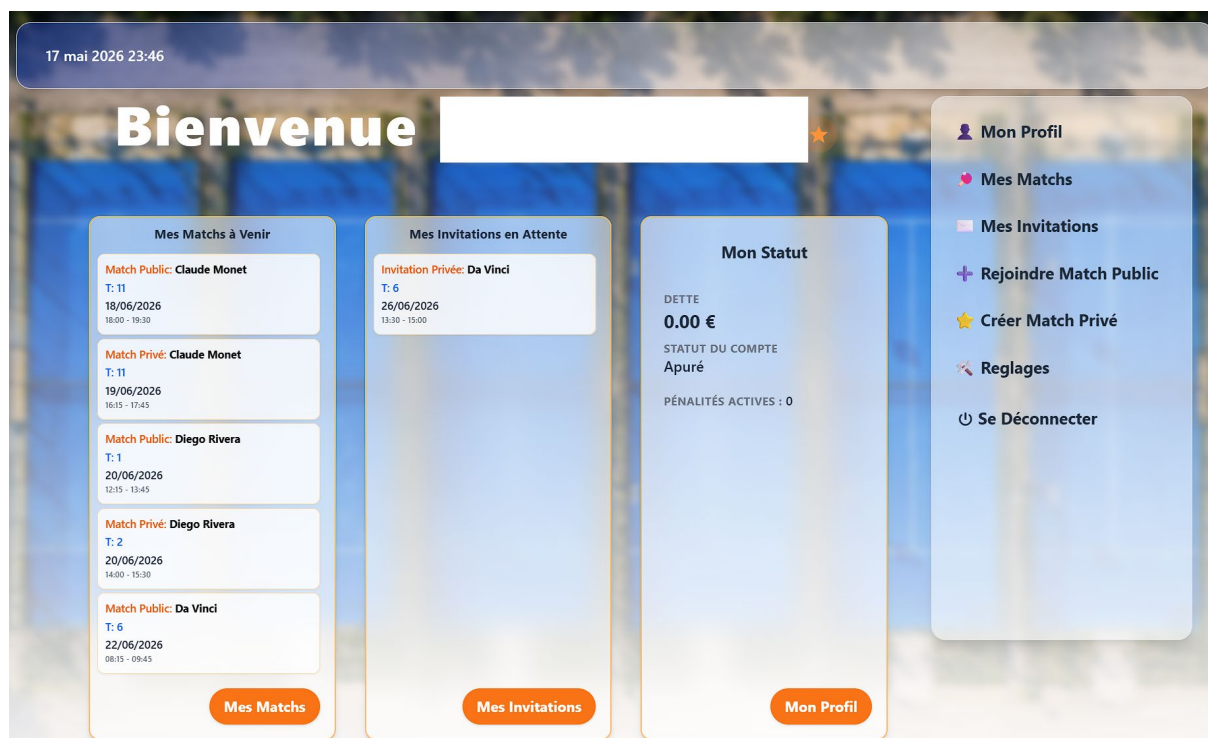


Figure 3.1 - Page utilisateur sur PC

- Sur Mobile (menu « hamburger ») :

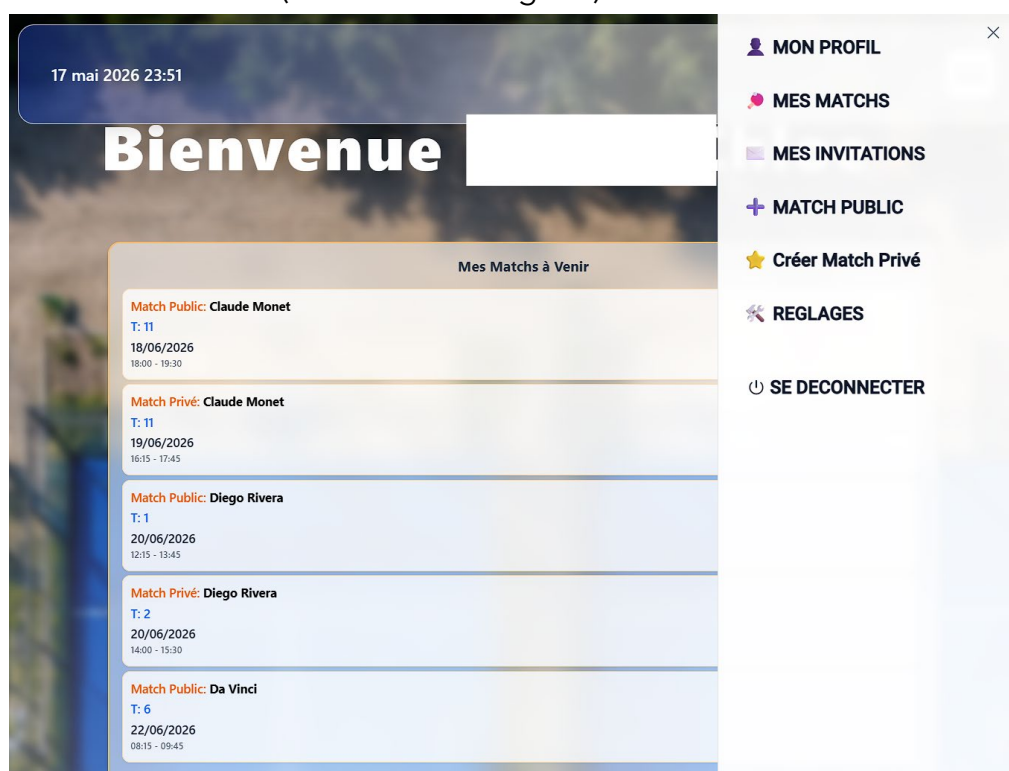


Figure 3.2 - Page utilisateur sur Mobile

2. Bouton « Créer Match Privé » :



3. Formulaire vierge de création d'événement privé :

Nouveau match privé organisé par [REDACTED]

Remplissez les informations ci-dessous pour créer un match privé.

Créer un Match

Complexe sportif

▼

Terrain

▼

Date

Heure du début

▼

Fin

--:--

Organisateur

[REDACTED]

Invités (veuillez inviter 3 joueurs)

Email de l'invité

Q

Email de l'invité

Q

Email de l'invité

Q

Créer le Match

Figure 3.3 - Formulaire de création de l'événement privé (vide)

4. Formulaire de création d'événement privé complété – étape d'invitation :

Nouveau match privé organisé par [REDACTED]

Remplissez les informations ci-dessous pour créer un match privé.

Créer un Match

Complexe sportif

Diego Rivera

Terrain

Field 3 - en Intérieur

Date

24/05/2026

Heure du début

15:45

Fin

17:15

Organisateur

[REDACTED]

Invités (veuillez inviter 3 joueurs)

[REDACTED]

✓

×

Utilisateur trouvé: (matricule: [REDACTED])

[REDACTED]

✓

×

Utilisateur trouvé: (matricule: [REDACTED])

utilisateurnonexistant@example.be

Q

×

INVITER ?

Aucun utilisateur trouvé pour cet email.

Créer le Match

Figure 3.4 – Formulaire de création de l'événement privé complet et invitation

5. Bouton « Inviter ? » :

Aucun utilisateur trouvé pour cet email.

6. Formulaire d'inscription - mot de passe requis (pas d'e-mail de bienvenue automatique) :

Inscription

Prénom *

Nom *

Email *

Mot de passe *

Confirmer le mot de passe *

Date de naissance *

Site *

Niveau *

Figure 3.5 - Formulaire inscription

7. Formulaire de Paiement :

Paiement

Montant à payer: **15 €**

Numéro de carte

1234567891234567

Annuler Payer

Figure 3.5 - Formulaire de paiement

8. Pop-up de confirmation :

Votre match du 24/05/2026 de 15:45 à 17:15 est réservé. Le paiement du créateur a été enregistré (CARD). Les 3 invités ont une demande de paiement en attente.

OK

Figure 3.6 - Pop-up de confirmation

9. Exemple d'une invitation reçue :

Mes invitations / Paiements en attente

Terrain: 6 → 26/06/2026 13:30:00 - 15:00:00

Organisateur,-trice: [REDACTED]

Montant: 15.00 €

Complexe: Da Vinci

Participants: 4 | En attente: 2

Accepter / Payer Décliner

Figure 3.7 - Invitation reçue

3.4. Architecture applicative

Les informations ci-dessous proviennent du dossier d'architecture. Consultez ce document pour consulter davantage de détails comme :

- La structure hiérarchique (folders) du projet.
- Les librairies, outils et frameworks structurants.
- La documentation de l'API (Swagger).

Ce document est disponible à la source du projet.

3.4.1. Architecture Backend

L'architecture backend suit un modèle **3-tier (3 couches)** avec une séparation stricte des responsabilités.

Hiérarchie des Couches :

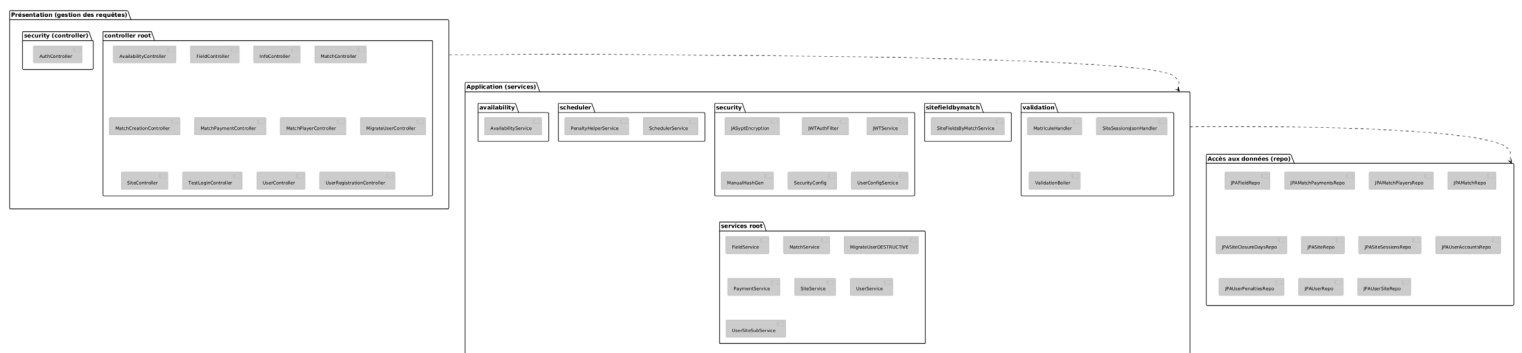


Figure 4.1 – Architecture Backend

Le schéma hiérarchique peut être consulté plus en détails sur [PlantUML](#).

Couche Présentation (Couche inférieure)

- Rôle : Gestion des requêtes HTTP entrantes.
- Responsabilités :
 - Réception des requêtes HTTP (REST API).
 - Appel du service approprié.
 - Gestion des autorisations (appel service JWT).
 - Formataje des réponses (JSON et codes HTTP).

- Technologies :
 - Spring Boot
 - Spring Doc OpenAPI
 - Spring Web

Couche Application / Services (*Couche intermédiaire*)

- Rôle : Contient la logique métier et l'orchestration.
- Responsabilités :
 - Ordonnancement des opérations.
 - Validation des règles.
 - Appel des repositories pour accéder aux données.
 - Vérification des autorisations (JWT et Spring Security)
 - Transformation des données (DTOs).
- Technologies :
 - Spring Boot
 - Spring Service
 - Spring Security
 - Spring Scheduler
 - JASypt
 - Jackson
- Tests :
 - JUnit
 - Javafaker

Couche Accès aux Données (Couche supérieure)

- Rôle : Interaction avec la base de données.
- Responsabilités :
 - Récupération des données.
 - Transformation des données (modèles).
 - Exécution des requêtes JPA/Hibernate.
- Technologies :
 - Spring Data JPA
 - Jdbc
 - Lombok
 - Hibernate
 - Microsoft SQL Server / Docker
- Tests :
 - H2 database

3.4.2.Architecture Frontend

L'architecture frontend suit un modèle **basé sur les composants** avec une séparation claire en **trois couches**.

Hiérarchie des Couches :

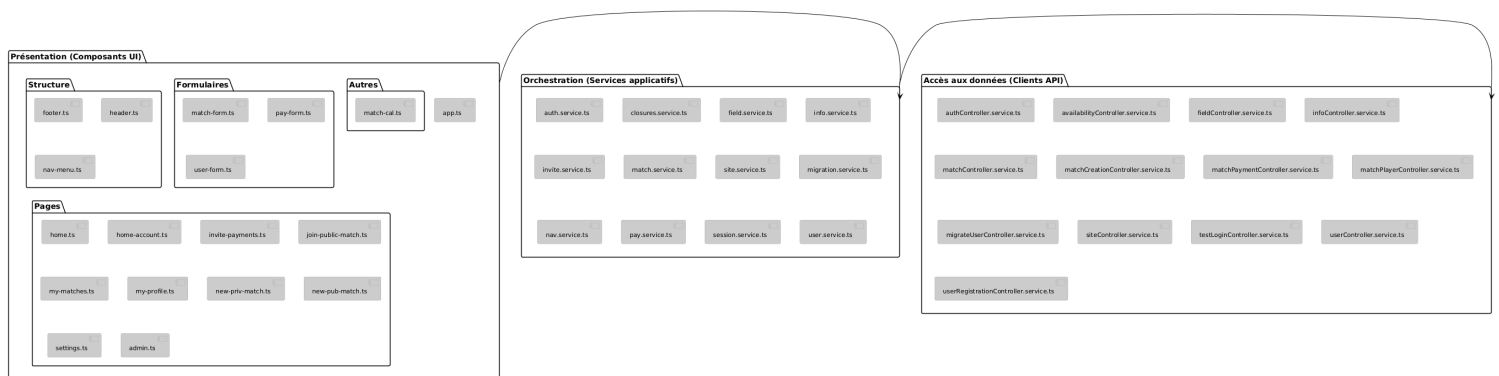


Figure 4.2 - Architecture Frontend

Le schéma hiérarchique peut être consulté plus en détails sur [PlantUML](#).

Couche Présentation (*Composants*)

- Rôle : Gestion de l'interface utilisateur et de l'expérience utilisateur.
- Responsabilités :
 - Affichage des données.
 - Gestion des interactions utilisateur (clics, formulaires, etc.).
 - Appel du service approprié.
 - Application des styles.
 - Logique de présentation (affichage conditionnel, animations).
- Technologies :
 - Angular
 - Tailwind CSS
 - Angular Material
 - RxJS
 - TypeScript
 - HTML5
- Tests :
 - Cypress
 - Karma-jasmine

Couche Service (*Orchestration*)

- Rôle : Coordination entre la couche de présentation et la couche de données.
- Responsabilités :
 - Appels aux API backend
 - Transformation des données
 - Gestion des erreurs
 - Logique métier côté client

- Technologies :
 - Angular Services
 - RxJS

Couche Accès aux Données (*Client HTTP*)

- Rôle : Communication avec le backend avec usage de Open API.
- Responsabilités :
 - Requêtes HTTP (GET, POST, PUT, DELETE).
 - Interception des requêtes/réponses (ajout de headers, gestion des erreurs globales).
 - Définition des modèles de données (interfaces TypeScript).
- Technologies :
 - Angular
 - Open API generator
 - HttpClient (Angular)

Extrait 1 - « Dossier d'architecture.pdf »

4. Contraintes fonctionnelles

4.1. Règles

Les règles de contraintes fonctionnelles, dont les exigences générales ont déjà été définies dans la section : analyse métier ([page 9](#)), ont pour objectif de contrôler et valider la logique métier. Afin d'en faciliter la consultation et l'application, ces règles sont structurées par catégories fonctionnelles et présentées sous forme de tableaux détaillés aux pages suivantes. Cette organisation permet de clarifier leur rôle et leur impact sur le système.

Les entités qui sont mentionnés dans les règles décrites dans ce chapitre peuvent être retrouvés dans le chapitre « Modélisation des données » ([page 45](#)).

Index des règles :

Accès et autorisation : [CF-001 à CF-005](#)

Authentification et sécurité : [CF-006 à CF-010](#)

Gestion des événements (matches) : [CF-011 à CF-024](#)

Gestion des utilisateurs et invitations : [CF-025 à CF-034](#)

Paielements et abonnements : [CF-035 à CF-040](#)

Consultation et affichage : [CF-041 à CF-044](#)

4.1.1.Accès et autorisation

ID	Description
CF-001	Un utilisateur, défini dans <i>Users</i> , ne peut accéder qu'aux fonctionnalités autorisées par son rôle (0 = Invité, 1 = Utilisateur Site, 2 = Utilisateur Tous Sites (VIP), 7 = Administrateur Site, 9 = Super Administrateur (Tous Sites). Les rôles sont définis dans <i>Users_Roles</i> .
CF-002	Un administrateur (rôle 7 ou 9) a un accès complet à la gestion du/des site(s) (selon le rôle), terrains, utilisateurs (selon le rôle) et matchs.
CF-003	Tout utilisateur doit être connecté (jeton JWT valide) pour accéder à la plateforme.
CF-004	La modification des sites, terrains et matchs est réservée uniquement aux administrateurs (rôle 7 ou 9).
CF-005	Un utilisateur ne peut modifier que son propre profil (sauf administrateur).

4.1.2. Authentification et sécurité

ID	Description
CF-006	L'authentification doit être sécurisée (mot de passe haché, JWT avec expiration).
CF-007	Un jeton d'authentification JWT expire après 900 secondes.
CF-008	La déconnexion doit invalider la session (suppression du JWT côté serveur).
CF-009	Le mot de passe doit respecter : 8 caractères minimum, 1 majuscule, 1 minuscule, 1 chiffre et un caractère spécial accepté (@, !, -, +, &, \$, €)
CF-010	Un utilisateur inactif (<i>Users.is_active</i> = 0) ne peut pas se connecter.

4.1.3. Gestion des événements (matches)

ID	Description
CF-011	Un match, défini dans <i>Matches</i> , doit comporter exactement 4 joueurs.
CF-012	Un match doit avoir une durée de 90 minutes (début + 90 min = fin).
CF-013	Les matchs consécutifs sur un même terrain (<i>Fields.field_id</i>) doivent être espacés d'au moins 15 minutes entre <i>Matches.end_time</i> et <i>Matches.start_time</i> .
CF-014	Un créneau tampon de 15 à 30 minutes doit être respecté avant le premier match (<i>MIN(Matches.start_time)</i>) et après le dernier match (<i>MAX(Matches.end_time)</i>) d'une journée sur un terrain.
CF-015	La création d'un match implique la création des entrées joueur correspondantes (<i>Match_Players</i>) et des paiements en attente (<i>Match_Payments</i>) dans le cas d'un match privé (<i>priv_status</i> != NULL).
CF-016	Un match doit être payé intégralement (<i>Match_Payments.status</i> = 'clear' chez tous les participants) pour être confirmé.
CF-017	Un match ne peut pas dépasser les horaires d'ouverture du terrain (<i>Sites_Sessions</i>).
CF-018	Tout match public se verra annulé si le nombre de joueurs n'est pas atteint.
CF-019	Un match ne peut pas être créé sur un terrain fermé (<i>Fields.is_active</i> = 0) ou en maintenance (<i>maintenance_from/to_date</i>), ou sur un site inactif (<i>Sites.is_active</i> = 0).

CF-020	Un match privé doit inclure 1 organisateur et 3 invitations définies dans <i>Match_Players</i> .
CF-021	Un match privé incomplet (moins de 4 joueurs confirmés) est converti en match public 48h avant la date du match.
CF-022	Si un match privé est converti en public, une pénalité est appliquée à l'organisateur pour défaut de joueurs (<i>Users_Penalties.reason = 'insufficient_players'</i>), pour une durée de 7 jours. Cette pénalité sera levée dès que le délai indiqué est passé.
CF-023	Un utilisateur ne peut rejoindre un match public que si des places sont disponibles (<i>Match_Players compte < Matches.max_players</i>).
CF-024	Un match public peut être créé par un administrateur (rôles 7,9) ou un utilisateur (rôles 0,1,2) selon les droits.

4.1.4. Gestion des utilisateurs et invitations

ID	Description
CF-025	Un utilisateur qui s'inscrit sur la plateforme doit fournir : prénom, nom, e-mail, mot-de-passe, date de naissance, site et le niveau.
CF-026	Un utilisateur avec un rôle inférieur (rôles 0 ou 1) peut mettre à jour son niveau d'abonnement.
CF-027	Chaque utilisateur se voit assigner un identifiant unique (matricule) correspondant au rôle occupé dans le système.
CF-028	Un intervenant inscrivant un nouvel utilisateur (à inscrire) doit fournir : prénom, nom, date de naissance et le niveau (pas de mot de passe, lien d'inscription envoyé par e-mail).
CF-029	Un e-mail d'invitation doit être valide (format RFC 5322).
CF-030	Un utilisateur ne peut pas être invité deux fois au même match.
CF-031	Un utilisateur ne peut pas s'inviter lui-même à un match (précision : l'organisateur est automatiquement assigné à son propre match privé et ne peut pas s'inviter lui-même.).
CF-032	Un utilisateur avec une pénalité active (<i>Users_Penalties.is_active = 1</i> ET <i>end_date > NOW()</i>) ne peut pas être invité.
CF-033	Un administrateur (rôles 7 ou 9) ne peut pas être invité à un match.
CF-034	Un utilisateur inactif (<i>Users.is_active = 0</i>) ne peut pas être invité.

4.1.5. Paiements et abonnements

ID	Description
CF-035	Un utilisateur doit avoir un solde suffisant sur son portefeuille (<i>Users_Accounts.balance</i>) ou sur son compte bancaire, ou en espèces lors d'une transaction comptoir, lors d'un paiement.
CF-036	La mise à niveau d'abonnement nécessite une méthode de paiement valide.
CF-037	Le paiement d'un match privé doit être effectué avant la date limite (48 heures avant <i>Matches.match_date</i>).
CF-038	Le paiement d'un match public doit être effectué avant la date limite (24 heures avant <i>Matches.match_date</i>).
CF-039	Si un match privé est converti en public, une dette est appliquée à l'organisateur : $balance = (balance - (\frac{1}{4} \text{ du prix total du match } * \text{ invités non-confirmés ou défaut de paiement}))$ ET <i>status = 'debt'</i> . Une raison est également spécifiée.
CF-040	Un utilisateur avec une dette en cours (<i>status = 'debt'</i>) se voit infliger une pénalité ' <i>reason = unpaid_balance</i> ' avec un délai d'expiration de 5 ans. Cette pénalité sera levée dès l'approvisionnement des fonds.

4.1.6. Consultation et affichage

ID	Description
CF-041	Un utilisateur ne peut consulter que ses matchs à venir (<i>Match_Players.user_id.match_id = Matches.match_id</i> ET <i>Matches.match_date > NOW()</i>).
CF-042	Un utilisateur ne peut consulter que ses propres invitations (<i>Match_Players.user_id = user_id</i> ET <i>status = 'pending'</i>).
CF-043	Les dates d'ouverture affichées doivent respecter les dates d'ouverture du site (<i>Sites_Closures</i>).
CF-044	Les créneaux horaires affichés doivent respecter les horaires d'ouverture du site (<i>Sites_Sessions</i>) et fermetures pour maintenance des terrains (<i>Fields</i>).

4.2. Calculs et opérations complexes

4.2.1. Calcul des sessions sur base d'horaires d'ouverture

Description Générale du Mécanisme

Le système génère dynamiquement les créneaux de réservation (sessions) pour un site en se basant sur ses heures d'ouverture et de fermeture. Il les calcule lors de la création pour créer un planning optimisé. Le résultat est une structure de données au format JSON qui contient la liste des sessions disponibles pour une journée type (chaque entrée JSON contient notamment start_time, end_time, ainsi que des métadonnées (match_set_id, duration_minutes)).

Ce mécanisme garantit que l'ensemble des sessions, pauses et temps tampons obligatoires s'intègrent parfaitement dans le temps d'ouverture total du site, en cherchant à maximiser le nombre de sessions réservables.

Constantes et Paramètres de Calcul

Le calcul repose sur un ensemble de constantes définies dans le code, qui dictent la structure de la journée :

- Durée d'une session : 90 minutes
 - Durée standard allouée pour un match.
- Pause entre les sessions : 15 minutes
 - Temps de battement entre la fin d'une session et le début de la suivante.
- Temps tampon post-session : 15 minutes
 - Temps obligatoire à la fin de la journée, après la dernière session, pour permettre aux joueurs de quitter les lieux. Cette contrainte garantit que la dernière session se termine au moins 15 minutes avant l'heure de fermeture du site.
- Temps tampon pré-session (variable pour optimisation) : Un choix stratégique entre 15 et 30 minutes.
 - Il est crucial de noter que cette valeur n'est pas un intervalle continu (par exemple, 20 ou 25 minutes ne sont pas utilisés). Le système teste deux scénarios distincts : l'un où la journée commence par un tampon de 15 minutes, et l'autre avec un tampon de 30 minutes. Il sélectionne ensuite l'option qui maximise le nombre total de sessions possibles ou, en cas d'égalité du nombre de sessions générées entre

les deux scénarios (pré-session 15 min vs 30 min), applique une règle secondaire basée sur le temps restant en fin de journée (préférence pour 30 min seulement si cela ramène le temps restant à ≤ 30 min alors qu'il dépasserait 30 min avec 15 min). C'est une optimisation par scénario, utilisant deux valeurs fixes et prédéfinies.

Validation de faisabilité des horaires

Avant de générer le planning, le système vérifie que la plage horaire d'ouverture du site est suffisante pour accueillir au moins une session complète en respectant les temps tampons et la durée de session. Si aucune session ne peut être placée (ex. plage trop courte), les horaires sont considérés comme non valides et la création / mise à jour des horaires est refusée.

En plus de la génération des sessions (qui réserve un temps tampon post-session fixe de 15 minutes avant la fermeture), le système applique une validation de cohérence sur les horaires du site. Cette validation considère qu'un temps tampon post-session doit rester raisonnable, et impose une borne supérieure (max 30 minutes) afin d'éviter des plages horaires trop longues laissant un temps inutilisé excessif après la dernière session. Ainsi, lors de la validation, le tampon post-session est encadré par : $15 \text{ min} \leq \text{post-session} \leq 30 \text{ min}$.

Équations et Logique de Calcul

a) Calcul du Temps Disponible

Le temps total d'ouverture est d'abord converti en minutes pour simplifier les calculs.

$$\text{`TempsTotalDisponible (min) = HeureFermeture - HeureOuverture`}$$

b) Calcul du Temps Utilisable pour les Sessions

Le temps réellement utilisable pour les sessions et les pauses est le temps total, moins les tampons fixes (pré-session et post-session).

$$\text{`TempsUtilisable (min) = TempsTotalDisponible - TempsTamponPréSession - TempsTamponPostSession`}$$

c) **Calcul du Nombre de Sessions**

Le nombre de sessions est calculé en divisant le temps utilisable par la durée d'un bloc "session + pause". On utilise la fonction `floor` (partie entière) car une session ne peut pas être partielle.

$$\text{BlocSession (min)} = \text{DuréeSession} + \text{PauseInterSession}$$

$$\text{NombreDeSessions} = \text{floor}((\text{TempsUtilisable} + \text{PauseInterSession}) / \text{BlocSession})$$

Note : On ajoute `PauseInterSession` au numérateur car la dernière session n'est pas suivie d'une pause, ce qui libère 15 minutes. C'est une manière mathématique d'exprimer que le nombre de pauses est `N-1` pour `N` sessions.

d) **Processus d'Optimisation**

Le cœur du mécanisme est le choix du Temps tampon pré-session optimal (15 ou 30 minutes). Le système procède comme suit :

- Simulation 1 : Il calcule le `NombreDeSessions` en utilisant un `TempsTamponPréSession` de 15 minutes.
- Simulation 2 : Il recalcule le `NombreDeSessions` en utilisant un `TempsTamponPréSession` de 30 minutes.
- Comparaison :
 - Si un scénario permet d'obtenir plus de sessions que l'autre, il est choisi.
 - Si les deux scénarios produisent le même nombre de sessions, le système applique une règle secondaire basée sur le temps restant après la dernière session : il peut préférer 30 min seulement si le temps restant avec 30 min est ≤ 30 min alors qu'il serait > 30 min avec 15 min ; sinon il conserve 15 min.

e) Génération de la Liste des Sessions

Une fois le `TempsTamponPréSession` optimal déterminé, le système génère la liste finale des sessions de manière itérative :

- La première session commence à `HeureOuverture + TempsTamponPréSessionOptimal`.
- Chaque session suivante commence à l'heure de fin de la précédente, plus la `PauseInterSession` de 15 minutes.
- Le processus s'arrête lorsque l'ajout d'une nouvelle session dépasserait le `TempsTotalDisponible` en tenant compte du `TempsTamponPostSession` obligatoire de 15 minutes.

4.2.2. Génération du matricule

Description Générale du Mécanisme

Le système attribue automatiquement un matricule à chaque utilisateur lors de sa création. Ce matricule sert d'identifiant interne lisible et permet notamment d'identifier rapidement la catégorie de l'utilisateur grâce à un préfixe lié au rôle.

L'objectif de ce mécanisme est de garantir un identifiant structuré et compréhensible, permettre de distinguer les catégories d'utilisateurs (ex. administrateurs vs utilisateurs standard) et d'assurer une numérotation progressive et cohérente au sein d'une même catégorie.

Structure du matricule

Le matricule est composé de deux parties :

- Un préfixe alphabétique dépendant du rôle de l'utilisateur (ex. lettre spécifique selon la catégorie).
- Une partie numérique avec un nombre de chiffres défini :
 - Administrateurs : 3 chiffres (ex. : A001)
 - Autres utilisateurs : 4 chiffres (ex. : L0001)

Méthode de génération

Le système identifie d'abord la catégorie de l'utilisateur (son rôle) afin de déterminer :

- Le préfixe à utiliser,
- Le nombre de chiffres (3 ou 4).

Ensuite, il consulte les matricules déjà existants qui partagent le même préfixe.

Deux cas sont possibles :

- a) Aucun matricule existant pour ce préfixe :
 - Le système initialise la série au premier numéro standard (001 ou 0001).
- b) Des matricules existent déjà :
 - Le système récupère le plus grand numéro utilisé pour ce préfixe et génère le suivant ($\text{max} + 1$).

Le matricule est finalement formaté pour respecter strictement le nombre de chiffres attendu.

Pourquoi cette approche fonctionne

- a) Elle garantit une progression naturelle des matricules à l'intérieur d'un même type d'utilisateur.
- b) Elle facilite la lecture : le préfixe indique immédiatement la catégorie, et la partie numérique assure l'unicité dans la série.
- c) Elle évite les matricules "aléatoires" difficiles à tracer ou expliquer.

Points d'attention

Le système suppose que les matricules existants respectent le format "préfixe + nombre". Si des données historiques sont mal formatées, elles peuvent perturber la détection du plus grand numéro.

Dans des cas de créations simultanées (plusieurs inscriptions en même temps), il faut veiller à garantir l'unicité au niveau de la base de données afin d'éviter qu'un même matricule soit attribué deux fois.

5. Modélisation des données

5.1. Description des entités

Entité	Description
Users_Roles	Définit les différents rôles que les utilisateurs peuvent avoir (ex. : Invité, Utilisateur Site, VIP, Administrateur).
Users	Stocke les profils de tous les utilisateurs du système, incluant leurs informations personnelles, rôle, niveau et informations d'authentification.
Users_Accounts	Suit le statut financier (solde) des comptes utilisateurs, incluant le statut (clear/debt) et l'historique des mises à jour.
Users_Penalties	Gère les pénalités imposées aux utilisateurs, avec la raison, la durée, et le statut actif/inactif.
Users_Sites	Associe les utilisateurs à leurs sites de référence, avec des indicateurs pour le site primaire et le statut VIP.
Sites	Contient les détails spécifiques à chaque site sportif (nom, adresse, horaires d'ouverture, statut actif/inactif).
Sites_ClosureDays	Enregistre les dates de fermeture partagées et spécifiques à chaque site (maintenance, événements spéciaux, jours fériés).
Sites_Sessions	Définit les créneaux horaires (sessions) pour les matchs sur chaque site, stockés sous forme de JSON.
Fields	Répertorie les terrains disponibles sur chaque site, avec leur statut (actif/inactif) et périodes de maintenance.
Matches	Enregistre tous les matchs, qu'ils soient privés ou publics, avec leurs détails (date, heure, organisateur, statut, prix).
Match_Payments	Journalise tous les paiements liés aux matchs, incluant le montant, la méthode de paiement, le statut et les dates.
Match_Players	Associe les joueurs à un match spécifique, avec leur rôle (p1, p2, p3, p4) et leur statut (approved, pending, declined).

5.2. Diagramme Entité-Association (EA)

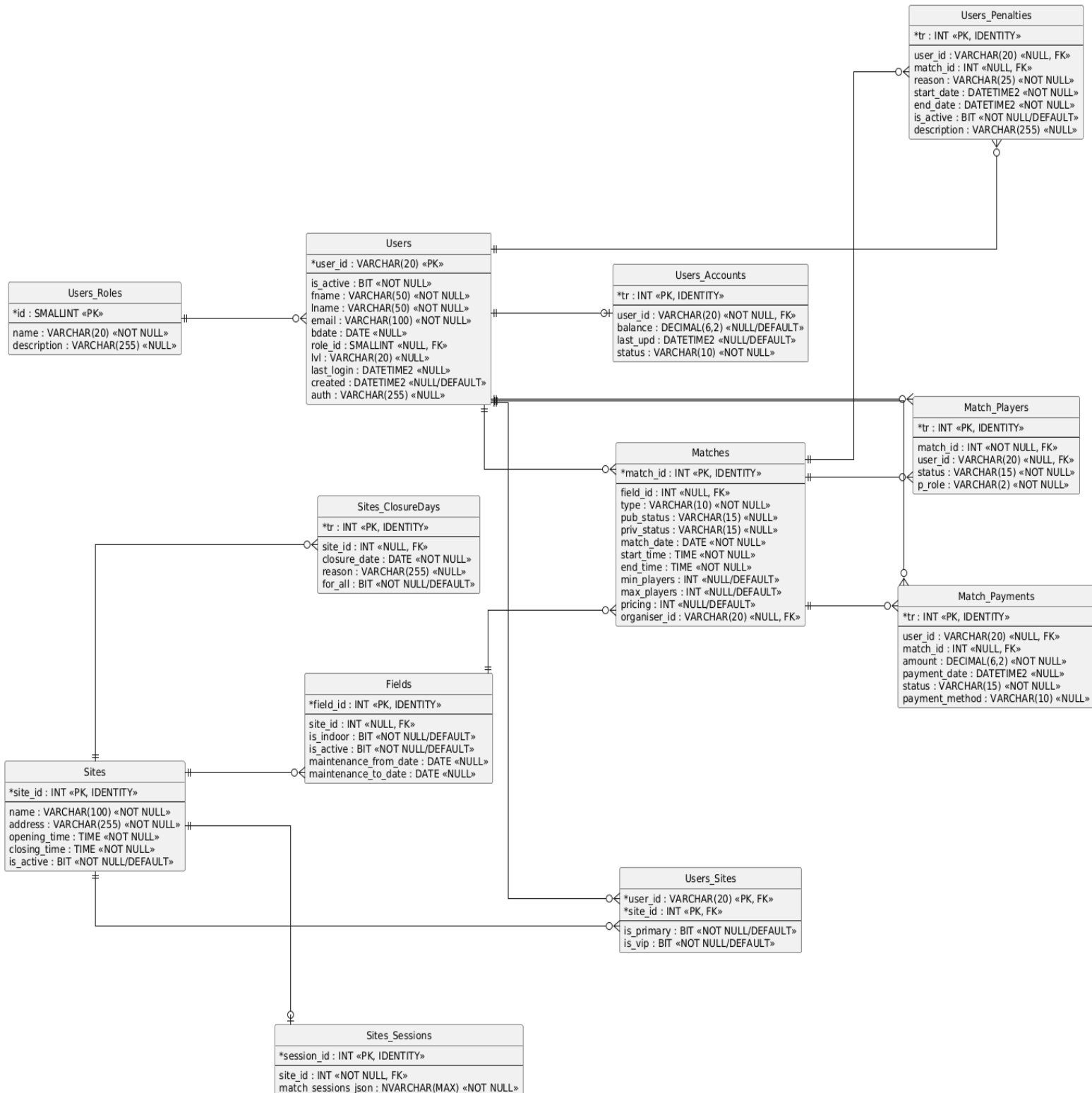


Figure 5 – Diagramme EA final base de données

Le diagramme entité-association peut être consulté plus en détails sur [PlantUML](https://plantuml.com/).

5.3. Diagramme relationnel

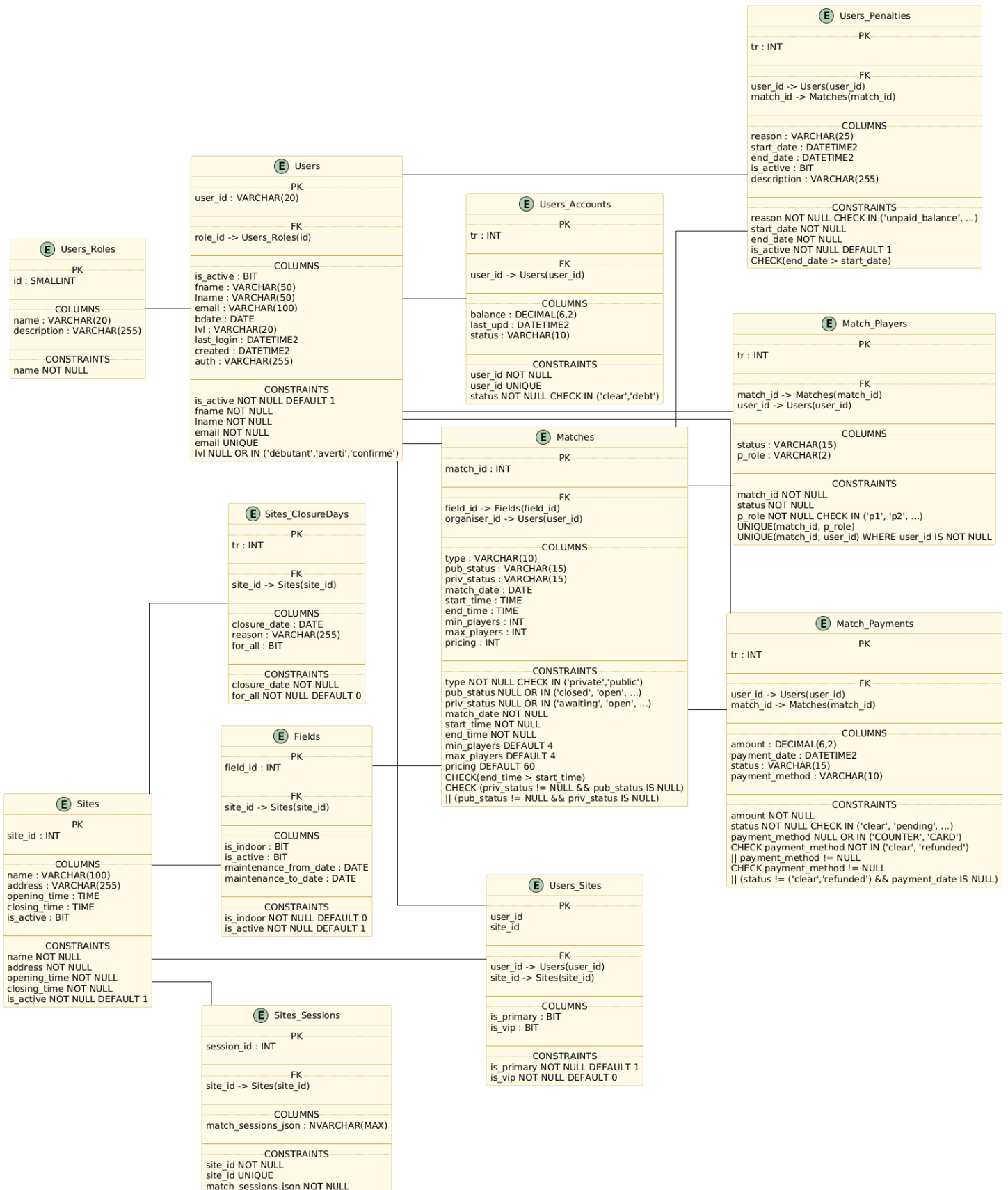


Figure 6 - Diagramme relationnel base de données

Le diagramme relationnel peut être consulté plus en détails sur [PlantUML](https://plantuml.com).

5.4. Processus de conception

5.4.1.Canvas d'analyse initiale

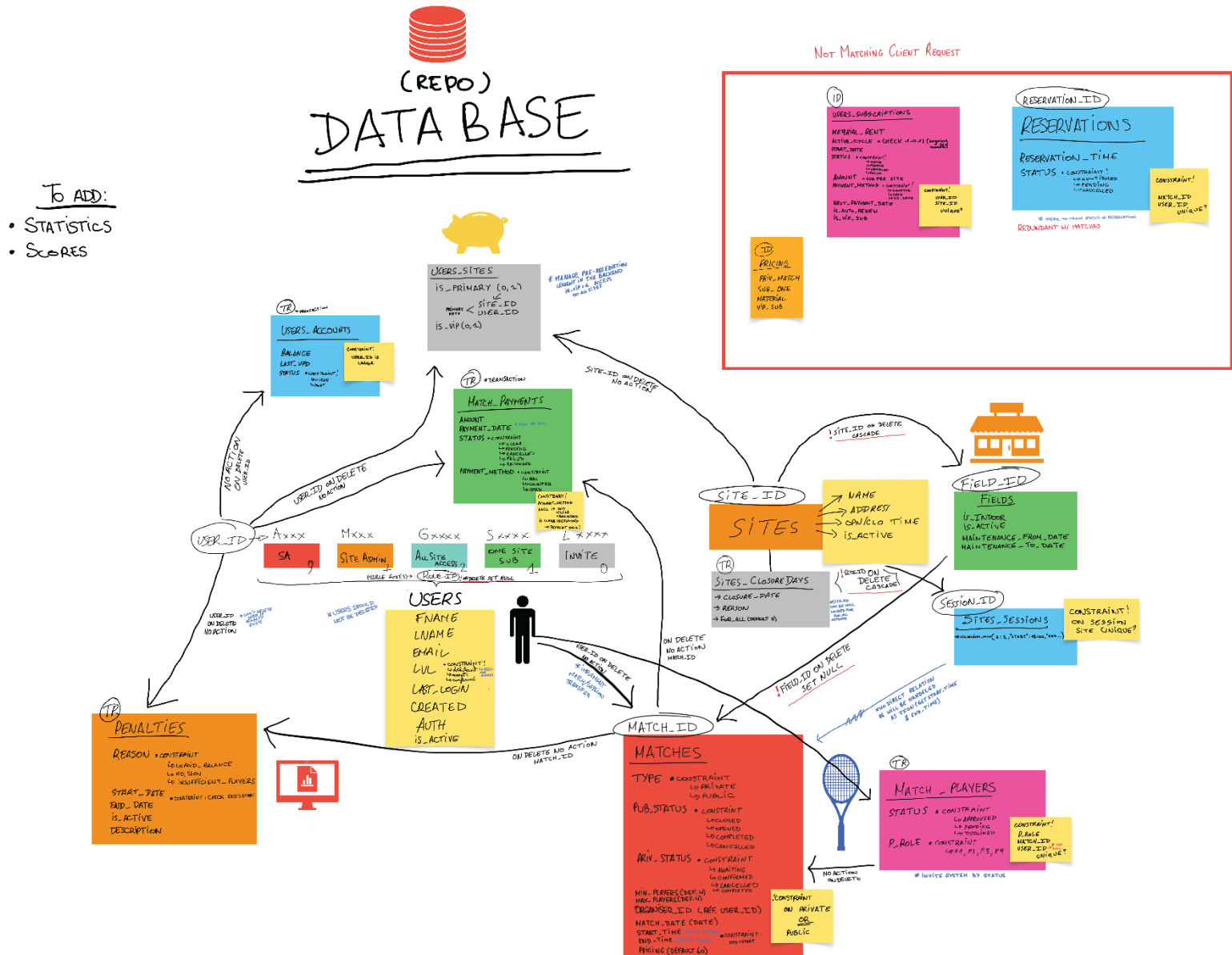


Figure 7 - Canvas EA base de données

5.5. Portée et évolution

5.5.1. Tableau d'implémentation des contraintes

Le tableau ci-dessous documente l'implémentation technique de chaque contrainte fonctionnelle définie en section 4.1 ([page 35](#)), en lien direct avec les attentes métier du projet.

Il permet de :

- Valider que toutes les règles de gestion sont effectivement appliquées dans le système.
- Localiser rapidement le code, la configuration ou la base de données associée à chaque contrainte.
- Garantir la traçabilité entre les exigences fonctionnelles et leur mise en œuvre technique (Backend, Frontend, Base de données).

Pour plus de détails, veuillez également vous référer aux exigences fonctionnelles ([page 9](#)).

Un index des contraintes pour rappel :

Accès et autorisation : [CF-001 à CF-005](#)

Authentification et sécurité : [CF-006 à CF-010](#)

Gestion des événements (matches) : [CF-011 à CF-024](#)

Gestion des utilisateurs et invitations : [CF-025 à CF-034](#)

Paielements et abonnements : [CF-035 à CF-040](#)

Consultation et affichage : [CF-041 à CF-044](#)

ID	Localisation précise & Mécanisme de validation
CF-001	<u>Backend</u> : backend/.../security/SecurityConfig.java (Règles d'API) interdisant globalement l'accès non authentifié. <u>Frontend</u> : frontend/src/app/app.routes.ts (Utilisation de Route Guards Angular) et masquage des composants de l'UI via la syntaxe @if dans les templates HTML (ex: app/layout/...). <u>DB (p3sgbd-prodv)</u> : Table Users_Roles définissant les niveaux d'accès.

CF-002	<u>Frontend</u> : frontend/src/app/app.routes.ts (Utilisation de Route Guards Angular isAdmin = [7, 9]) et masquage des composants de l'UI via la syntaxe @if dans les templates HTML.
CF-003	<u>Backend</u> : Filtre JWT (backend/.../security/JWTAuthFilter.java) interrogeant le SecurityContext pour valider le jeton avant l'accès aux routes sécurisées. <u>Frontend</u> : frontend/src/app/app.routes.ts (Redirection des non-authentifiés vers root « / »). : frontend/src/app/core/... (Intercepteur HTTP ou mécanisme d'injection du JWT dans le header Authorization).
CF-004	<u>Frontend</u> : frontend/src/app/app.routes.ts (Utilisation du authGuard Angular bloquant strictement l'accès aux URLs de création/modification pour le rôle < 7) et masquage des composants de l'UI via la syntaxe @if.
CF-005	<u>Backend</u> : backend/.../controller/UserController.java > updateUser() (Le service vérifie que l'ID de la cible correspond au profil JWT connecté). <u>Frontend</u> : frontend/src/app/app.routes.ts (Vérification dans le authGuard via if (currentMatricule === requestedId)).
CF-006	<u>Backend</u> : backend/.../security/SecurityConfig.java (Déclaration et utilisation de BCryptPasswordEncoder pour hacher/comparer les mots de passe). <u>DB (p3sgbd-prodv)</u> : Table Users contient les mots de passe hachés.
CF-007	<u>Backend</u> : Fichier backend/.../application.properties (jwt.expiration=900) et classe utilitaire JWT listant l'expiration.
CF-008	<u>Backend</u> : backend/.../controller/security/AuthController.java > logout() (Invalidation de la session logicielle globale). <u>Frontend</u> : frontend/src/app/layout/content/header/header.ts (Méthode supprimant la session/localStorage et redirigeant le client vers root « / »).
CF-009	<u>Backend</u> : backend/.../controller/UserRegistrationController.java (Logique de validation manuelle via java.util.regex.Pattern compilant les regex de format (mot de passe fort, nom/prénom) avant persistance). <u>Frontend</u> : frontend/src/app/features/auth/ (Utilisation des Validators.pattern, Validators.required, et Validators.email dans les Reactive Forms Angular avant même d'interroger le backend).
CF-010	<u>Backend</u> : backend/.../services/validation/ValidationBoiler.java (verifyUserActive) & UserService interdisant la connexion si Users.is_active = 0.
CF-011	<u>Backend</u> : backend/.../services/MatchService.java > newMatch() (Exige explicitement le passage de 4 identifiants de joueurs valides). <u>DB (p3sgbd-prodv)</u> : Table Match_Players. Les contraintes check_player_role (p1 à p4) et uq_match_role (unicité du rôle par match) garantissent nativement au niveau de la DB qu'un match ne peut excéder 4 joueurs distincts.

CF-012	<u>Backend</u> : backend/.../services/validation/ValidationBoiler.java (Calcul écartant toute demande où l'heure de fin n'est pas "heure de début + 90 min"). <u>DB (p3sgbd-prodv)</u> : Table Matches. La contrainte check_match_time (end_time > start_time) empêche l'insertion de créneaux temporels négatifs ou nuls.
CF-013	<u>Backend</u> : backend/.../services/validation/SiteSessionsJsonHandler.java (Définit en "dur" les écarts en constantes SESSION_DURATION_MINUTES = 90 et BREAK_MINUTES = 15 lors du calcul de génération de la DB des créneaux du site).
CF-014	<u>Backend</u> : backend/.../services/validation/SiteSessionsJsonHandler.java (Algorithme calculateOptimalPreSession optimisant les espaces tampon entre 15 et 30 min (MIN/MAX_PRE_SESSION_MINUTES) autour des heures d'ouverture/fermeture).
CF-015	<u>Backend</u> : backend/.../services/MatchService.java > newMatch() (Génère automatiquement les entrées Match_Players et Match_Payments en cascade).
CF-016	<u>Backend</u> : backend/.../services/scheduler/SchedulerService.java (Tâche @Scheduled confirmMatchPayment() scrute les inscriptions. Si un participant n'a pas finalisé via backend/.../services/PaymentService.java, le processus de confirmation est bloqué). <u>DB (p3sgbd-prodv)</u> : Table Match_Payments. La définition stricte de la contrainte chk_payment_status agit comme verrou final sur les états financiers acceptés (clear, pending, etc.).
CF-017	<u>Backend</u> : backend/.../services/validation/ValidationBoiler.java (Croise les horaires du match avec les plages allouées dans DB > Sites_Sessions).
CF-018	<u>Backend</u> : backend/.../services/scheduler/SchedulerService.java (Tâche @Scheduled annulant les matchs publics où les joueurs manquent).
CF-019	<u>Backend</u> : backend/.../services/validation/ValidationBoiler.java > verifyFieldNotUnderMaintenance() inspectant Fields.is_active et les dates.
CF-020	<u>Backend</u> : backend/.../services/MatchService.java > L.224 (newMatch() vérifie que l'organisateur + la liste d'hôtes équivaut à 4 profils distincts). <u>DB (p3sgbd-prodv)</u> : Table Match_Players. Les contraintes check_player_role (p1 à p4) et uq_match_role (unicité du rôle par match) garantissent nativement au niveau de la DB qu'un match ne peut excéder 4 joueurs distincts.
CF-021	<u>Backend</u> : backend/.../services/scheduler/SchedulerService.java (Tâche planifiée déclassant les matchs privés si Match_Players confirmés < 4 à H-48).
CF-022	<u>Backend</u> : backend/.../services/scheduler/SchedulerService.java (Applique l'insertion dans Users_Penalties avec reason = 'insufficient_players').
CF-023	<u>Backend</u> : backend/.../services/MatchService.java > fetchAvailablePublicMatches() (Exclut de l'affichage les matchs via le compteur approvedCount < 4).

CF-024	<u>Backend</u> backend/.../controller/MatchCreationController.java (Endpoint global /api/creatematch injectant le type 'public' si l'utilisateur en possède les droits d'accès initiaux). <u>Frontend</u> : frontend/src/app/app.routes.ts (Route Guard bloquant explicitement l'URL 'create_public' pour restreindre les rôles < 7). : frontend/src/app/layout/content/nav-menu/nav-menu.ts (Conditionnement de l'UI générant la route create_public ou create_pmatch selon qu'un profil est Admin ou Standard).
CF-025	<u>Backend</u> : backend/.../controller/security/AuthController.java (Utilise une DTO stricte exigeant tous les champs au /register). <u>DB (p3sgbd-prodv)</u> : Table Users. Le niveau est figé via la contrainte check_users_lvl ('débutant', 'averti', 'confirmé').
CF-026	<u>Backend</u> : backend/.../services/UserService.java (Autorise expressément l'update du seul champ niveau pour les profils basiques). <u>Frontend</u> : masquage des composants de l'UI via la syntaxe @if dans les templates HTML (ex: app/layout/...).
CF-027	<u>Backend</u> : backend/.../services/UserService.java (Logique de création d'identifiant concaténant le numéro de rôle utilisant le service MatriculeHandler).
CF-028	Voir CF-025
CF-029	/
CF-030	<u>Backend</u> : backend/.../services/MatchService.java (Filtrage des collections d'invités pour bloquer les ID dupliqués). <u>Frontend</u> : Utilisation des Validators.pattern, Validators.required, et Validators.email dans les Reactive Forms Angular avant même d'interroger le backend.
CF-031	<u>Backend</u> : backend/.../services/MatchService.java > newMatch() (L'organisateur étant p1, il est impossible d'ajouter l'ID orga dans les invités p2, p3, p4). <u>Frontend</u> : Utilisation des Validators.pattern, Validators.required, et Validators.email dans les Reactive Forms Angular avant même d'interroger le backend).
CF-032	<u>Backend</u> : backend/.../services/validation/ValidationBoiler.java > verifyNoActivePenalties() (Rejette les invitations si pénalité détectée).
CF-033	<u>Backend</u> : backend/.../services/validation/ValidationBoiler.java > verifyNotAdminUser() (Empêche d'ajouter un rôle 7 ou 9 dans la liste des joueurs).
CF-034	<u>Backend</u> : backend/.../services/validation/ValidationBoiler.java > verifyUserActive() (Bloque l'invitation pour tout joueur désactivé).
CF-035	/
CF-036	/
CF-037	<u>Backend</u> : backend/.../services/scheduler/SchedulerService.java (Vérifie le timer à H-48 spécifiquement pour les matchs de type = private).

CF-038	<u>Backend</u> : backend/.../services/scheduler/SchedulerService.java (Vérifie le timer à H-24 spécifiquement pour les matchs de type = public).
CF-039	<u>Backend</u> : backend/.../services/scheduler/SchedulerService.java (Tâche @Scheduled convertAwaitingPrivateMatchesScheduledForTomorrow(). En cas d'échec privé, déduit le 1/4 du total par utilisateur non confirmé pour créer une dette chez l'organisateur). <u>DB (p3sgbd-prodv)</u> : Tables Users_Accounts et Users_Penalties. Le statut de la dette et la raison de la pénalité sont verrouillés par les contraintes check_user_status et check_penalty_reason.
CF-040	<u>Backend</u> : backend/.../services/scheduler/SchedulerService.java (Tâche @Scheduled applyPenaltiesForDebtors() qui applique formellement dans Users_Penalties la raison unpaid_balance jusqu'en NOW() + 5 ans). <u>DB (p3sgbd-prodv)</u> : Tables Users_Accounts et Users_Penalties. Le statut de la dette et la raison de la pénalité sont verrouillés par les contraintes check_user_status et check_penalty_reason.
CF-041	<u>Backend</u> : backend/.../services/MatchService.java > fetchMyUpcomingMatches() (Filtre SQL et Java retenant uniquement Matches.match_date > NOW()). <u>Frontend</u> : frontend/src/app/features/matches/ ou calendar/ (Composants UI Angular filtrant visuellement les créneaux via des boucles @for et empêchant la sélection des cases grisées/indisponibles signalées par le Backend).
CF-042	<u>Backend</u> : backend/.../services/MatchService.java (Requête affiliée sur la DB Match_Players couplé au compte utilisateur filtrant sur status = 'pending').
CF-043	<u>Backend</u> : backend/.../services/validation/ValidationBoiler.java (Interroge les intervalles de la DB Sites_ClosureDays avant d'autoriser la vue / l'action).
CF-044	<u>Backend</u> : backend/.../services/availability/AvailabilityService.java (Construit les créneaux disponibles conditionnés par horaires Sites_Sessions et entretien Fields). <u>Frontend</u> : frontend/src/app/features/matches/ ou calendar/ (Composants UI Angular filtrant visuellement les créneaux via des boucles @for et empêchant la sélection des cases grisées/indisponibles signalées par le Backend).

5.5.2.Stratégie d'implémentation des contraintes

L'analyse de l'intégration des contraintes (CF-X) démontre une conception multicouche robuste. Près de 80% des règles métiers complexes : comme le respect des espacements horaires (CF-013), l'annulation automatique par lot (CF-018), ou le routage selon les rôles (CF-004) – sont exécutées de manière centralisée par un Backend applicatif lourd développé en Java/Spring Boot (services, validation, schedulers, etc.).

Toutefois, la base de données relationnelle (p3sgbd-prodv) ne se contente pas d'être un simple espace de stockage persistant. Elle joue activement le rôle de dernier gardien de l'intégrité (Data Integrity) via de multiples contraintes de type CHECK et UNIQUE.

Par exemple, pour les contraintes CF-011 et CF-020 exigeant un maximum de 4 joueurs par match, la base de données couple un CHECK (p_role IN ('p1', 'p2', 'p3', 'p4')) avec un index d'unicité UNIQUE (match_id, p_role). Ce couplage garantit que, même en cas de contournement ou de bogue au niveau du Backend, le schéma empêchera physiquement l'insertion d'un 5ème joueur ou d'un doublon de rôle. Cette délégation ciblée assure une propreté stricte des données métier critiques (prix, temps, statuts de paiement et pénalités).

5.5.3.Fonctionnalités reportées

Les fonctionnalités suivantes ne sont pas incluses dans la version actuelle du système et ne sont pas reprises dans le diagramme entité-association ([page 46](#)) pour des raisons de priorisation et de contraintes temporelles, mais sont planifiées pour les versions futures :

Fonctionnalité	Raison de l'exclusion	VC	Impact technique	P
Statistiques (ex. : nombre de matchs par site, revenu par site, etc.)	Développement non critique pour le MVP.	v.2	Ajout de tables dédiées (ex. : <i>Match_Stats</i> , <i>Revenue_Stats</i>) ou utilisation de requêtes SQL simples pour extraire les données directement depuis <i>Matches</i> , <i>Users_Accounts</i> , et <i>Sites</i> (ex. : COUNT, SUM, GROUP BY).	1

Durées admises de pré-réservation par rôle.	Actuellement implémenté au niveau service. En cours d'analyse pour une intégration directe dans la base de données pour une meilleure cohérence et maintenabilité.	v.2	Ajout d'une table <i>Users_Booking_Rules</i> (avec champs : <i>role_id</i> , <i>allowed_duration</i>) et adaptation des services existants.	1
Scores (ex. : suivi des points par équipe/joueur)	Non essentiel pour le cœur métier (réservations et paiements). Nécessite une refonte partielle de <i>Matches</i> et une gestion des dépendances complexes.	v.2 , v.3	Ajout de tables <i>Match_Scores</i> (pour les résultats de matchs) et <i>Player_Stats</i> (pour les statistiques individuelles), ou extension de <i>Matches</i> avec des champs <i>home_score/away_score</i> .	3

Justification :

La version actuelle se concentre sur les fonctionnalités critiques pour le lancement.

- Gestion des utilisateurs, matchs et paiements (MVP).
- Respect des contraintes temporelles du projet (délai imposé).

Les fonctionnalités exclues ont été évaluées et priorisées selon :

1. Valeur métier (ex. : les statistiques sont utiles mais pas bloquantes).
2. Complexité technique (ex. : les scores nécessitent une refonte partielle de la base de données).
3. Délai de livraison (focus sur le MVP pour une sortie rapide).

6. Conclusion

Ce projet de gestion de terrains de Padel représente une solution intégrée qui dépasse largement le cadre d'un simple système de réservation. Il englobe une logique métier complexe : gestion multisites, plannings dynamiques, fermetures exceptionnelles, hiérarchies d'utilisateurs, rôles administratifs, ainsi que les flux événementiels (matchs publics/privés), de paiements et de pénalités.

Les livrables produits,

- Diagramme des cas d'utilisation.
- Diagramme Entité-Association.
- Diagramme relationnel.
- Le tableau d'implémentation des contraintes.

Ont permis de :

- 1.** Illustrer les interactions entre acteurs et fonctionnalités (diagramme des cas d'utilisation).
- 2.** Structurer les concepts métiers et leurs relations (diagramme Entité-Association).
- 3.** Concrétiser cette analyse en un schéma de base de données robuste et normalisé (diagramme relationnel).

Cette démarche a mis en lumière le rôle central de la base de données, qui ne se limite pas au stockage, mais garantit activement l'intégrité des données via des contraintes (`CHECK`, `UNIQUE`), des déclencheurs et une architecture en couches (appliquée aux Frontend, Backend et Base de données).

Réalisé avec une approche « Database-First », cette expérience démontre la compréhension des compétences clés de la formation : modélisation relationnelle, implémentation de contraintes et conception d'une architecture technique cohérente.

En définitive, une base de données bien conçue est le fondement d'une application robuste, maintenable et alignée avec les exigences métier.

6.1. Annexes

Projet cible : ProjetDV2026-PMALIOUKOV

Dépôt git : [ProjetDV2026_PMLKV](#)

Dossier d'architecture : [Dossier d'architecture.pdf](#)

Document d'exploitation (démarrage) : [README.md](#)

Auteur : Paul Malioukov (PSR10219)

Date : 18 mai 2026

Version : 3.0